To install sequelize with typescript and postgresql:

- Run command: *npm i pg sequelize*
- Run command: *npm i --save-dev @types/node typescript ts-node ts-node-dev*
    - Note: *ts-node-dev* is like *nodemon*, but for typescript
- Then we init the ts:
    - Run command: *npx tsc --init*

****

Then for the tsconfig.json we set:

```
"rootDir": "./",
"moduleResolution": "node10",
"outDir": "./dist",
```

****

For package.json scripts:

```
"scripts": {
    "dev" : "ts-node-dev --respawn src/server.ts",
    "build": "tsc",
    "start": "node dist/src/server.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
```

****

To run migrations, we must have *config/config.json*

****

The model file is:

```
import { DataTypes, Model } from "sequelize";
import db from "../config/database.config";

interface TodoObject {
    id: number,
    title: string,
    completed:boolean
}

export class Todo extends Model<TodoObject> {
}

Todo.init({
    id: {
        type: DataTypes.UUIDV4,
        allowNull: false,
        primaryKey: true,
        defaultValue: DataTypes.UUIDV4,
    },
    title: {
        type: DataTypes.STRING,
        allowNull: false,
    },
    completed: {
        type: DataTypes.BOOLEAN,
        allowNull: false,
        defaultValue: false,
    }
}, {
    timestamps: true,
    sequelize: db,
    tableName: "todos",
})
**********************************************************************************
```

To create todo:

```
try{
        const todo = await Todo.create({ ...req.body })
        return res.json({ todo: todo }).status(201)
    } catch (e) {
        return res.json({ err: e }).status(500)
    }
```

****

To get the todos using limit and offset:

```
router.get('/get-all-todos', async (req: Request, res: Response) => {

  try {

    let { limit, page } = req.query
    let size, offset
    if (limit == undefined) {
      size = 3;
    } else {
      size = parseInt(limit.toString())
    }

    if(page == undefined) {
      offset = 0
    } else {
      offset = (parseInt(page.toString()) - 1) * size;
    }

    let todos = await Todo.findAll({
      limit: size,
      offset: offset
    })

    return res.json({
      res: todos
    })
  } catch (e) {
    return res.json({ err: e }).status(500);
  }

});
```

****

To validate body or params of request using express-validator:

```
import { body, param, query } from 'express-validator/';

class TodoValidator {
    validateTodoReq() {
        return [
            body('title').notEmpty().withMessage('title is required'),
            body('title').isLength({ min: 2, max: 255 }).withMessage('title
length must be between 2 and 255'),
            body('completed').optional().isBoolean().withMessage('completed must
be true or false')
        ]
    }

    validatePaginationQuery() {
        return [
            query('limit').optional().isInt({ min: 1, max:
100}).withMessage('limit must be number and between 1 && 100'),
            query('page').optional().isInt({ min: 1 }).withMessage('page must be
number and between 1 && 100')
        ]
    }

    validateIdParam() {
        return [
            param('id').notEmpty().withMessage('id is
Required').isUUID(4).withMessage('id must be valid UUIDV4'),
        ]
    }
}

export default new TodoValidator();
```

****

To make middleware for errors of express-validator:

```
import { NextFunction, Request, Response } from "express";
import { validationResult } from "express-validator/"; // here we must add /

class CheckErrors {
  public checkAllErrors(req: Request, res: Response, next: NextFunction) {
    const errors = validationResult(req);

    if (!errors.isEmpty()) {
      return res.status(400).json({ errors });
    }

    next();
  }
}

export default new CheckErrors();
```

****